Homography & Perspective Warp

Week 11 — Session 2: Rectification & Auto-Alignment (RANSAC)

Practical Image Processing

Lecture 11.2 ▪ 60–70 min

Recap of Session 1

- Transforms move *positions*; the hierarchy runs rigid $\rightarrow$ affine $\rightarrow$ projective.
- A **homography** H is 3×3, 8 DoF, defined by **4 correspondences**.
- Apply it in homogeneous coords, then *divide by* w' — the perspective step.

Today: make it a scanner

Corners $\rightarrow$ destination rectangle $\rightarrow$ `warpPerspective` $\rightarrow$ flat scan, then do it *automatically* with matches + RANSAC.

Roadmap

Rectification: the DocScan core

Interpolation & sampling

Automatic alignment with RANSAC

In-class exercise

Deep dive & limits

Pitfalls & good practice

Wrap-up

Rectification: the DocScan core

The rectification idea

1. Find the 4 corners of the page/meter in the photo (Week 9's `approxPolyDP`, or feature matching).
2. Decide where those corners *should* go: the 4 corners of a clean rectangle.
3. Compute the homography from photo-corners $\rightarrow$ rectangle-corners.
4. `warpPerspective` the photo through it: out comes a flat, front-on scan.

Key idea

Rectification is just "I know where these 4 points *should* land" turned into a warp.

Step 1–2: a tilted meter and its 4 corners



the clean template (target)



simulated angled photo; 4 corners marked (red)

The corners are the correspondences: where each red dot *is* vs where it *should* be.

Step 3–4: warp to a flat scan



input: angled photo



output: rectified, front-on

`H = getPerspectiveTransform(corners, rectangle)`; `warpPerspective(photo, H, size)`.
Digits are now readable.

Getting the 4 corners in the right order

- `approxPolyDP` gives 4 points but in *arbitrary* order.
- Order them: top-left, top-right, bottom-right, bottom-left — else the warp flips/twists.
- Trick: TL has the smallest $x+y$; BR the largest; TR the smallest $x-y$; BL the largest.

✕ Pitfall

Wrong corner order = a mirrored or rotated scan. Always sort the corners consistently.

Worked example: order these 4 corners

Detected (arbitrary order):

Point	x	y	$x+y$
A	90	300	390
B	120	40	160
C	470	360	830
D	500	60	560

- TL = min $x+y$ $\Rightarrow$ B.
- BR = max $x+y$ $\Rightarrow$ C.
- Of the rest: TR = min $x-y$. $x-y$: A$= -210$, D$= 440$ $\Rightarrow$ TR = D.
- BL = the leftover $\Rightarrow$ A.

✓ Good practice

Order (B, D, C, A) = (TL, TR, BR, BL).

Choosing the destination rectangle

- You are free to pick any clean rectangle; match the object's true aspect ratio.
- A4 paper is $1 : \sqrt{2} \approx 1{:}1.414$. Pick width W; height $= W \times 1.414$.
- A utility meter faceplate might be nearly square, or wider than tall — measure it once.

Math

For $W = 800$ and A4: destination $= (0,0), (800,0), (800,1131), (0,1131)$, since $800 \times 1.414 \approx 1131$.

Interpolation & sampling

Inverse warping: every output pixel gets a value

- `warpPerspective` walks the *output* grid, maps each pixel back through H^{-1}, and samples the source.
- The back-mapped location is almost never an integer — it lands *between* source pixels.
- So we must **interpolate** a value from the neighbouring source pixels.

Key idea

Geometry says *where* to look; interpolation decides the *value* at that fractional spot.

Interpolation: nearest, bilinear, bicubic

- **Nearest**: copy the closest pixel — blocky but fast; correct for label/mask maps.
- **Bilinear**: weighted blend of the 4 surrounding pixels — smooth, the sensible default.
- **Bicubic**: 16-pixel fit — sharper edges, slower; nice for final high-quality output.

Note

`cv2.INTER_LINEAR` for photos, `INTER_NEAREST` for segmentation labels, `INTER_CUBIC` for a crisp final scan.

Worked example: bilinear at a fractional point

Back-mapped location $(x, y) = (2.25,\ 0.75)$ lands inside these four source pixels:

100	180
60	220

top-left 100, top-right 180, bottom-left 60, bottom-right 220

Math

$a = 0.25,\ b = 0.75$. Blend along x then y:

$$\text{top} = 100(0.75) + 180(0.25) = 120$$
$$\text{bot} = 60(0.75) + 220(0.25) = 100$$
$$v = 120(0.25) + 100(0.75) = 105$$

Nearest would have snapped to a single corner (e.g. 60) — bilinear gives the smoother 105.

Automatic alignment with RANSAC

From matches to a homography

- Instead of hand-picking 4 corners, use Week 10's feature matches.
- But matches contain wrong ones — a plain least-squares fit would be ruined by outliers.
- **RANSAC**: repeatedly fit H from 4 random matches, keep the H that most matches agree with.

Math

RANSAC = RANdom SAmple Consensus. It finds the transform supported by the largest set of **inliers**.

RANSAC, step by step

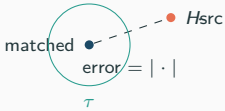
1. Pick 4 random correspondences; compute a candidate H.
2. Count **inliers**: matches whose points H maps to within a few pixels.
3. Repeat many times; keep the H with the most inliers.
4. Refit H using *all* its inliers for a precise result.

Key idea

RANSAC tolerates a large fraction of wrong matches — it only needs 4 good ones to exist.

Reprojection error: what “agree” means

- Push each source point through the candidate H; compare to its matched point.
- Distance below a threshold τ (say 3 px) $\Rightarrow$ **inlier**; else **outlier**.
- The best H maximises inlier count; `ransacReprojThreshold` sets τ.



How many iterations? (a quick estimate)

Math

With inlier fraction w and sample size s=4, the chance all 4 are inliers is w^4. To succeed with probability p:

$$N = \frac{\log(1-p)}{\log(1-w^4)}$$

- Example: $w = 0.6,\ p = 0.99 \Rightarrow w^4 \approx 0.13,\ N \approx \dfrac{\log 0.01}{\log 0.87} \approx 33$ iterations.
- Worse matches (smaller w) blow up N fast — filter matches first (ratio test) to raise w.

Automatic rectification on the real meter



angled photo (input)



auto-rectified via ORB + RANSAC

No manual corners: 66 ORB matches $\rightarrow$ 57 RANSAC inliers $\rightarrow$ `findHomography` $\rightarrow$ flat scan.

In-class exercise

In-class exercise (10 min)

Design the rectifier

1. You have the 4 page corners in the photo. What are the destination points for an A4-shaped scan of width 1000?
2. You resize a segmentation mask through the same warp. Which interpolation, and why?
3. Your automatic pipeline has 200 feature matches, 60 of them wrong. Which step saves you?
4. Roughly how many RANSAC iterations for $w = 0.7,\ p = 0.99$? (order of magnitude is fine)

Debrief

- (1) $(0,0), (1000,0), (1000,1414), (0,1414)$ — height $= 1000 \times 1.414$.
- (2) `INTER_NEAREST`: labels are categorical, blending would invent nonexistent class IDs.
- (3) **RANSAC** inside `findHomography` — it ignores the 60 outliers and fits to the inliers.
- (4) $w^4 \approx 0.24$, $N \approx \log(0.01)/\log(0.76) \approx 17$ — a couple dozen tries.

Key idea

Corners (or matches) + destination rectangle + homography + warp = a scanner in four lines of code.

Deep dive & limits

Planar assumption & its limits

- A homography is exact only for a *flat* surface (or a camera that only rotates).
- A curved page, a bent meter cover, or 3D objects break the model.
- For a gently curved document you need **dewarping** (a nonlinear grid) — beyond a single H.

✕ Pitfall

If your "scan" still bows at the edges, the surface wasn't flat — one homography can't fix curvature.

Other uses of homography

- **Panorama stitching**: warp each photo into a common frame and blend.
- **Augmented reality**: place a graphic onto a detected planar marker.
- **Bird's-eye view**: rectify a road/parking scene to a top-down map.
- **DocScan**: our meter/plate rectifier.

Key idea

Any time the world of interest is (locally) a plane, a homography is the right tool.

The whole rectifier in code

```
# corners: 4 page corners found in the photo (ordered TL,TR,BR,BL)
W, H = 800, int(800*1.414)
dst = np.float32([[0,0],[W,0],[W,H],[0,H]])
M = cv2.getPerspectiveTransform(corners, dst)
scan = cv2.warpPerspective(photo, M, (W, H),
                           flags=cv2.INTER_CUBIC)
# automatic variant: ORB matches -> cv2.findHomography(..., cv2.RANSAC)
```

Four lines turn an angled photo into a flat scan — the heart of DocScan.

The automatic variant, annotated

```
orb = cv2.ORB_create(2000)
k1,d1 = orb.detectAndCompute(template, None)
k2,d2 = orb.detectAndCompute(photo, None)
matches = matcher.match(d1, d2)              # some are wrong
src = np.float32([k1[m.queryIdx].pt for m in matches])
dst = np.float32([k2[m.trainIdx].pt for m in matches])
H, mask = cv2.findHomography(dst, src, cv2.RANSAC, 3.0)
print("inliers:", int(mask.sum()), "/", len(matches))
scan = cv2.warpPerspective(photo, H, template.shape[1::-1])
```

`mask` flags the inliers; the printed ratio is your confidence the alignment is good.

Pitfalls & good practice

Common pitfalls

✗ Too few or degenerate points

Three collinear "corners" can't define H. Ensure the 4 points form a real quadrilateral.

✗ Loose RANSAC threshold

A large τ lets outliers count as inliers and drags H off. Start around 3 px and tighten.

Good practice

✔ Filter matches before RANSAC

A ratio test (Week 10) raises the inlier fraction w, which slashes the iterations N needed.

✔ Always report the inlier ratio

High inliers/total $\Rightarrow$ trust the scan. A low ratio is your early warning to re-shoot.

Wrap-up

What you should now be able to do

- Rectify a tilted document: order corners, pick a destination rectangle, build H, `warpPerspective`.
- Choose the right interpolation for photos vs. label maps.
- Align automatically from feature matches with `findHomography` + RANSAC, and read the inlier ratio.
- Recognise when the planar assumption breaks and dewarping is needed.

This week's deliverable

Due before Week 12 (see `project/`)

1. Take/simulate an angled photo of a meter or document; detect its 4 corners and order them.
2. Rectify to a flat scan with `warpPerspective`; try nearest vs. bilinear and compare.
3. Also rectify automatically via ORB matches + `findHomography(RANSAC)`; report inliers/total.
4. Reflect: after rectification, why is digit reading (Weeks 12–14) much easier?

A homography turns a snapshot into a scan.

Four corners, one 3×3 matrix, one warp — and the page lies flat.